

Q: Any questions or
comments?

Lecture 6: Name Services and DNS

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

Contents

- Names and name services
 - & Directory services
- DNS
 - Logical model
 - Applications
 - Administration
 - Operation
 - Complications

Name Services (and Directory Services)

Names

- A **name** uniquely distinguishes one thing from other like it
 - E.g. a computer, service, remote object, file, user
- **Identifiers** are names intended for machine use
 - Contrast “human-readable” names
- **Descriptions** are statements about some aspect of a thing
 - May – or may not – be useful to identify a thing
 - See directory services

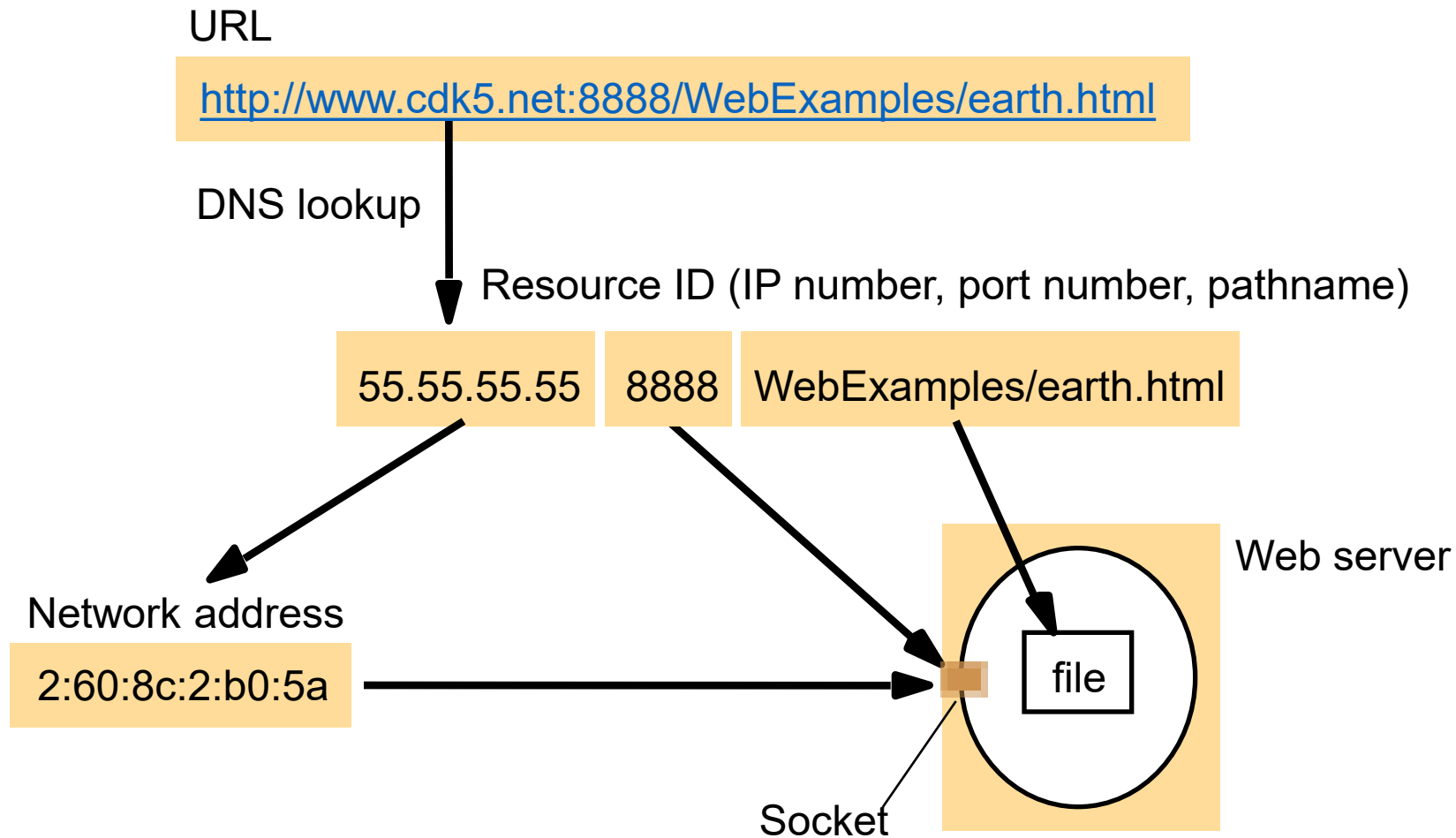
Names in distributed systems

- A **pure name** is (just) an array of bits
 - No structure or meaning – it just identifies
- An **address** specifically **locates** a resource
 - Good for access;
 - Bad if the object can move (the address is no longer valid)
- Different **services** may use different names
 - E.g. print service names printers, file service names directories and files
 - => **service-specific** names

URIs, URNs, URLs

- **Uniform Resource Identifiers (URIs)** are service-independent names
 - Defined for the World-Wide Web
- Uniform Resource **Names (URNs)** are pure names
 - So must always be looked up before they can be used, e.g. DOIs
- Uniform Resource **Locators (URLs)** include information to locate and access a resource
 - E.g. (next page)

Various naming domains used to access a resource from a URL



Name Services

- In a **name service**, names are **bound** to **attributes** of the object/resource
 - A name is **resolved** to get these attribute values
 - e.g. to get the address of the host it is on
- A **name space** is the set of possible names in a name service
 - Name spaces may be **flat** or **hierarchical**
 - Any given name may be **bound** or **unbound**
 - Name spaces can be subdivided into naming **contexts**
 - E.g. nodes within a hierarchical name space such as folders
- Name services can be independent of the distributed system
 - May contain names from different systems (unification)
 - May contain names from different administrative domains (integration)

Example: JavaRMI registry

see also previous lecture and lab on JavaRMI

- The **rmi registry** provides the **name service**
 - Can be run on each node on a well-known port (1099)
 - By default any process on the local machine can register an object
 - Or each server makes its own internal registry on its own known port (better in docker, e.g. in lab 3)
- **Names** are of the form “//HOST:PORT/OBJECT”
 - I.e. a fixed two-level hierarchical **name space**
 - First level is host name (and port)
 - Can be regarded as the host-specific context
 - (defaults to localhost:1099)
 - Second is object name on that host
- **Specific to Java RMI**

Directory Services

- **Directory services** allow clients to look up names or addresses using more flexible **queries**
 - E.g. SQL-like queries over **attribute** values
- Also known as
 - **yellow pages** services
 - attribute-based name services
- No globally used approach or system (contrast DNS)
 - E.g. X.500 OSI directory service
 - **LDAP** = simplified X.500 protocol over TCP/IP, widely used for intranet directory services, e.g. authentication on CS Linux machines)
 - E.g. Microsoft Active Directory
 - used by UoN for users & computers, with Microsoft Entra ≈ cloud AD

Q: How much do you know
about how DNS works?

DNS – Domain Name System

Note, DNS is both really important in most Internet-based systems, and a really good example of a global-scale distributed system!

Logical model: DNS name space

- Names in DNS are called **domain names**
- Name space is **hierarchical**, comprising
 - **Name components** or **labels** (not case sensitive)
 - And delimiters, i.e. “.”
 - E.g. “www.cs.nott.ac.uk”
- **Absolute**, not relative
 - But default domains are often configured to try as a possible suffix for unqualified names
- Attributes are encoded as **Resource Records** of various types for various purposes...

Applications:

DNS Resource Records

- **Host name resolution**, i.e. mapping a domain name to an IP address
 - DNS **A** (IPv4) and **AAAA** (IPv6) record
 - Attribute value is an IP address
 - DNS **CNAME** record creates an **alias**
 - Attribute value is the “real” (canonical) domain name
 - E.g. “www.cs.nott.ac.uk” -> “red.cs.nott.ac.uk”
 - Automatically checked when resolving a domain name to an IP address
- **Mail host location**, i.e. finding mail server to send internet email to
 - DNS **MX** records
 - Attribute value is the mail server’s domain name and a priority
- DNS implementation - NS and SOA records – see later slide

Q: how have you used
DNS?

E.g. DNS data records

Domain	TTL	class	type	value
nottingham.ac.uk.	300	IN	A	10.156.8.6
nottingham.ac.uk.	300	IN	A	185.18.139.133
nottingham.ac.uk.	300	IN	MX	10 nottingham-ac-uk.mail.protection.outlook.com.
nottingham.ac.uk.	300	IN	TXT	"v=spf1 mx ip4:212.53.87.53 ip4:54.229.2.165 ip4:52.30.130.201 ip4:82.201.24.52 ip4:72.5.51.49 include:spf.mail.wpmhost.net include:servers.mcsv.net include:legendware.co.uk include:mail-eu.geckoform.com include:spf01.nottingham.ac.uk. ~all"
...				
nottingham.ac.uk.	3600	IN	NS	uidapdns02.nottingham.ac.uk.
nottingham.ac.uk.	3600	IN	NS	uidapdns01.nottingham.ac.uk.
nottingham.ac.uk.	3600	IN	SOA	uidapdns02.nottingham.ac.uk. dnsadmin.nottingham.ac.uk. 2021296251 10800 3600 2419200 900

Other (less important) applications...

- **Reverse resolution**, i.e. mapping IP address to domain name
 - Uses DNS **PTR** records under the domain *in-addr.arpa*
 - Sometimes used for security
- **Host information**, e.g. operating system
 - Uses DNS **HINFO** records
 - Not often used due to security concerns
- **Service** lookup, e.g. in Bonjour/DNS-SD (see lecture 16 discovery)
 - Uses DNS **SRV** records -> hostname **and** port number
- Other **arbitrary information**, e.g. various anti-email spam standards and domain ownership tests
 - Uses DNS **TXT** records
 - (originally used to hold system information but that was insecure)

Multiple records / values

- A single domain name can (usually) have multiple records of the **same** type
= alternative values
 - usually for **redundancy**/reliability and/or **load balancing**
- E.g.
 - Multiple NS records => duplicate nameservers
 - Multiple MX records => alternative mail servers/relays
 - Multiple A records => multiple network interfaces for same host, or multiple hosts providing the same service
 - E.g. www.bbc.co.uk (but we'll also look at some other options for scaling in lecture 9)
 - (NOT multiple CNAME records, because there can only be one “canonical” name)
- Returned in different orders to different clients
 - By default client takes the first value
 - Can get and use others, e.g. if the first fails
 - MX records have an explicit preference value to guide selection order

Administration:

Administrative domains

- An **administrative domain** is a subset of namespace with a single administrative authority
 - E.g. a single organisation
- Corresponds to a domain name suffix
 - Called DNS **zone**
 - E.g. “nottingham.ac.uk” -> University of Nottingham
 - Sub-domains can be **delegated** to other authorities
- Each **authority** has complete control over its own administrative zones
 - Including delegation to other authorities
 - NB: avoids centralised management

Registering a Domain Name

Q: who has their own domain?

- IANA decides which **top-level domains** can exist
 - E.g. .com, .uk
- A number of approved organisations (**registrars**) can assign sub-domains to customers
 - E.g. .nottingham.ac.uk -> the University of Nottingham
- Some specialised domains/sub-domains are managed by a single organisation or registrar with special rules
 - E.g. .gov (US government), .gov.uk (UK government), .ac.uk (JISC)
- The customer can either keep all their DNS records on the registrar's DNS servers (e.g. with a web management interface), or run their own...

Operation:

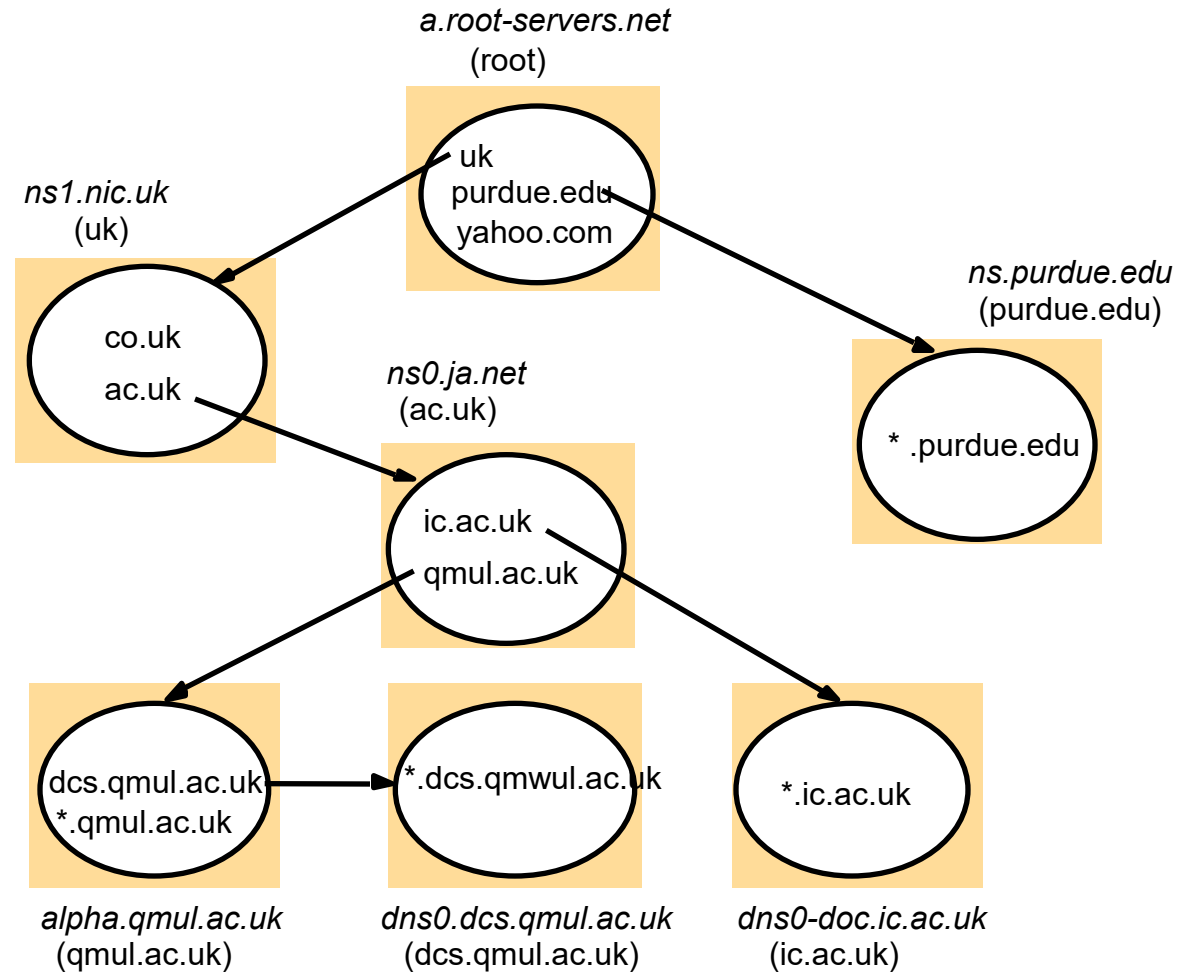
DNS Name servers

- A DNS **server** is responsible for one (or more) **zones**
 - Gives **authoritative answers** for those zones (contexts)
 - Must be **replicated** (at least 2 failure-independent servers)
 - Each zone/server is identified in DNS by a **SOA** (Start Of Authority) record
 - **NS** (Name Server) records identify responsible (authoritative) name servers
 - Including delegation of a sub-zone to another name server, see next slide
- A **root server** gives answers for “.”, the **root** domain
 - Highly replicated
 - Must know at least all top-level domain servers
 - And often much more

DNS name servers example

Note: Name server names are in italics, and the corresponding domains are in parentheses.

Arrows denote name server entries



Name resolution

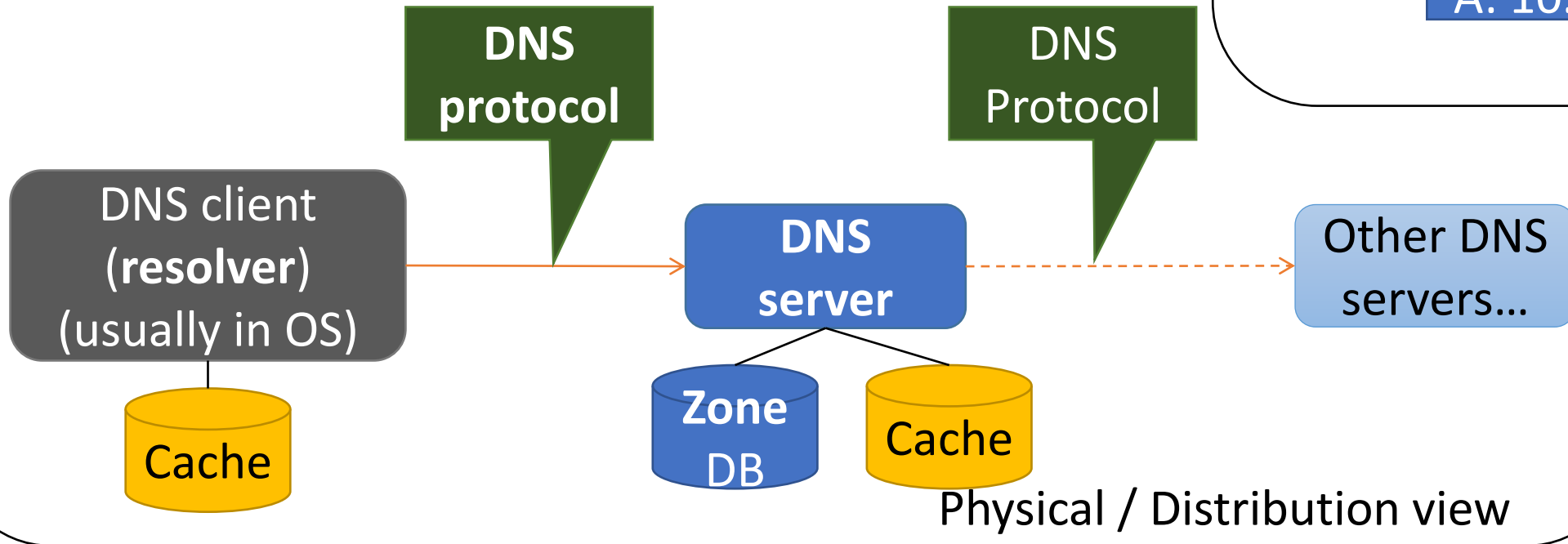
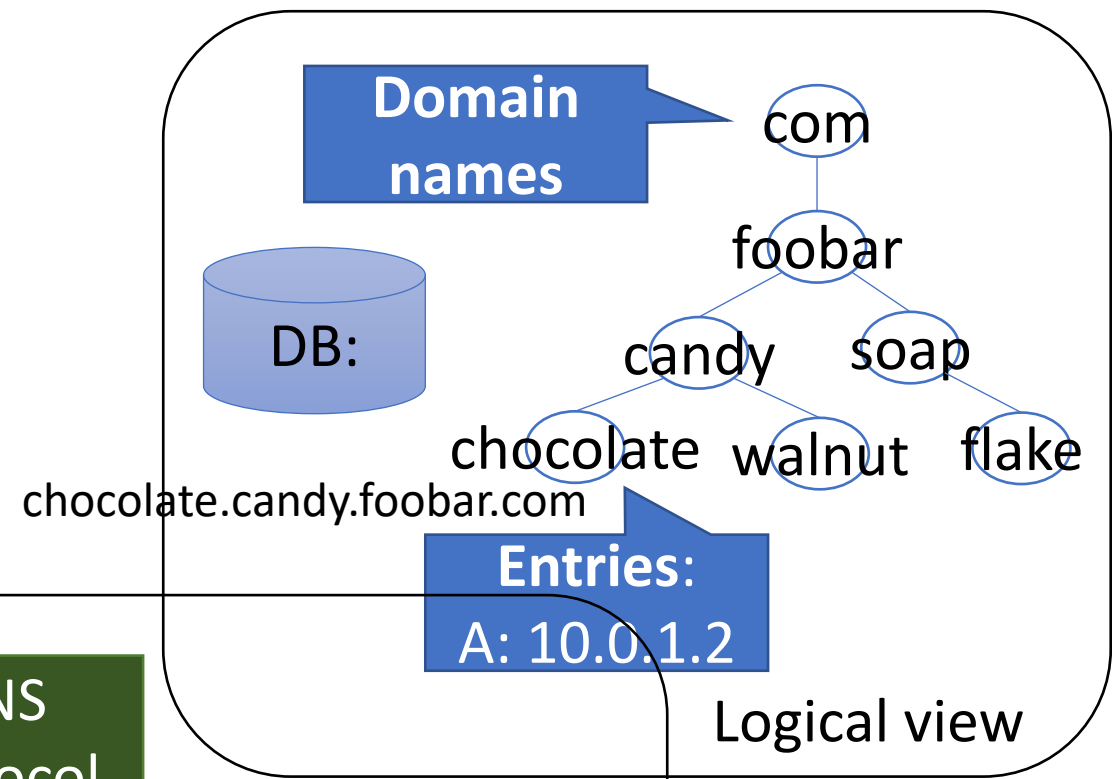
- DNS name resolution is performed by a **resolver**
 - Sends initial request to a pre-configured name server, e.g. learnt through DHCP
- Resolution may take multiple steps
 - The (first) name server may not know the answer
 - But knows the address of a different name server to ask
 - E.g. a root server if going “up” the tree,
 - or a delegated sub-zone name server if going “down” the tree
- These steps may be handled
 - **Iteratively**: the resolver asks each name server in turn, or
 - **Recursively**: the first server asks the second, and so on

Cacheing

Q: What is cacheing?

- Every resource record has a **Time To Live (TTL)**
 - = How long the value can be cached (often 1 hour)
 - Large value => more cacheing
 - => reduced load on servers
 - but slower response to updates
- **Cacheing** is critical to the performance of DNS
 - Note, applications often look up the same domain name repeatedly, e.g. browsing a web site
 - Resolvers cache recent responses
 - Servers cache recent responses to recursive queries
- Cached responses are **non-authoritative**
 - i.e. may be out of date, c.f. authoritative responses from the zone server

DNS - summary



Complications:

Other notes on DNS

- DNS records may be updated automatically = **Dynamic DNS (DDNS)**
 - E.g. as a computer joins a network
 - Uses a mixture of proprietary approaches and standards
- DNS is rather insecure
 - e.g. some attacks give clients the IP address of a malicious server in place of the real server's IP address
 - **Secure** extensions for DNS (**DNSSEC**) are not universally available/used
 - **DNS over HTTPS** (DoH) is a (controversial) proposed standard for browsers (in particular) to do DNS over HTTPS to trusted DoH servers
- DNS can be done by **multicast** without a server (i.e. **mDNS**)
 - See later lecture on multicast & discovery

More complications with DNS

- DNS is also used to exfiltrate data or as a hidden control channel
 - which is why access to external DNS servers is **blocked** by the University firewall
- **Docker** provides a small built-in DNS server (on `127.0.0.11`)
 - Containers on a custom network use the embedded docker DNS server
 - Returns local IP addresses for local containers (as in labs)
 - Forwards other requests to the host's configured DNS server
- **systemd-resolved** (part of systemd on many Linux distros) also acts as a local DNS server (usually on `127.0.0.53`)
 - Can be configured to look up and pass on queries in various ways, e.g. DNSSEC, DNS over TLS

Name Services Summary

- A **name** distinguishes one particular thing from others like it
 - **Identifiers** are names typically intended for machine use
- A **name service** allows a (pure) **name** to be **resolved** to an **address** or other **attributes** bound to that name
 - e.g. to contact the named resource
 - The set of possible names constitute the **name space**, which is often a **hierarchy**
- WWW **Uniform Resource Identifiers (URIs)** include
 - Uniform Resource **Locators (URLs)**
 - Uniform Resource **Names (URNs)**, which are pure names
- A **directory service** allows resources to be looked up more flexibly than a name service
 - Using **queries** based on attribute values.

DNS Summary

- **DNS** is a global name service used in the Internet
 - Supports **host name resolution** (using A records), **mail host location** (using MX records), service **aliases** (using CNAME records) and various other applications on the Internet
- DNS is divided into **administrative domains** or **zones**
 - Each is **independently controlled** and can be partially **delegated**
 - Each is supported by a replicated **name server**
 - **Root** servers handle the root domain
- DNS clients use a **resolver** to resolve domain names
 - May require repeated queries (**iteratively** or **recursively**) to obtain an answer
- DNS Includes **cacheing** in resolvers and servers for scalability and performance

Next steps

- Lecture 7: Q&A (& past papers?)
- Lab 4: DNS
- Lecture 8: World Wide Web